Advanced Empirical Finance: Topics and Data Science

Transaction costs and parametric portfolio policies

Stefan Voigt

University of Copenhagen and Danish Finance Institute (DFI)

Spring 2027

From Monday, every trade costs

- **Trading game:** from 1 March, every trade costs 10 bps (0.1 %) of its value
- Turnover per month of the strategies of lecture 4: plug-in minimum variance 61%, Ledoit-Wolf 23%, no short sales 16%, 1/N 4%

> **Today:** how trading costs change the optimal portfolio, and a portfolio rule with a handful of parameters for thousands of stocks

Transaction costs

What trading costs

- **Proportional costs:** spreads, fees, commissions. A trade of $|\Delta\omega_i|$ costs $c\,|\Delta\omega_i|$ of wealth, e.g. c = 10 bps in the trading game
- **Price impact:** large trades move the price; the cost grows faster than the trade. A quadratic cost $\frac{\beta}{2}\Delta\omega_i^2$ is a common approximation
- **Weights drift:** even without trading, prices move the weights. Before rebalancing, the portfolio holds

$$\omega_{t^+} = \frac{\omega_t \circ (1 + r_t)}{\iota'\big(\omega_t \circ (1 + r_t)\big)}$$

so the trade is $\omega_{t+1} - \omega_{t^+}$, and the turnover is $\sum_i |\omega_{i,t+1} - \omega_{i,t^+}|$

Lecture 4's strategies after costs

Strategy	Sharpe ratio, no costs	Sharpe ratio, 10 bps	Sharpe ratio, 50 bps	Turnover per month
Buy and hold	1.05	1.05	1.05	0%
1/N	0.96	0.95	0.94	4%
Minimum variance, no short sales	0.86	0.85	0.79	16%
Ledoit-Wolf minimum variance	0.90	0.87	0.79	23%
Plug-in minimum variance	0.78	0.74	0.55	61%

- Dow Jones stocks, 60-month windows, 2005–2025, excess returns net of proportional costs on the turnover
- The more a strategy trades, the more costs take away: 1/N gains on the minimum-variance portfolios
- **Buy and hold** starts with 1/N in February 2005 and never trades. It lets the winners grow, and today's Dow Jones stocks are the winners we know today: look-ahead bias in the choice of stocks (lecture 3)

Whiteboard: quadratic transaction costs

Mean-variance with a cost for moving away from the current weights (Hautsch & Voigt, 2019):

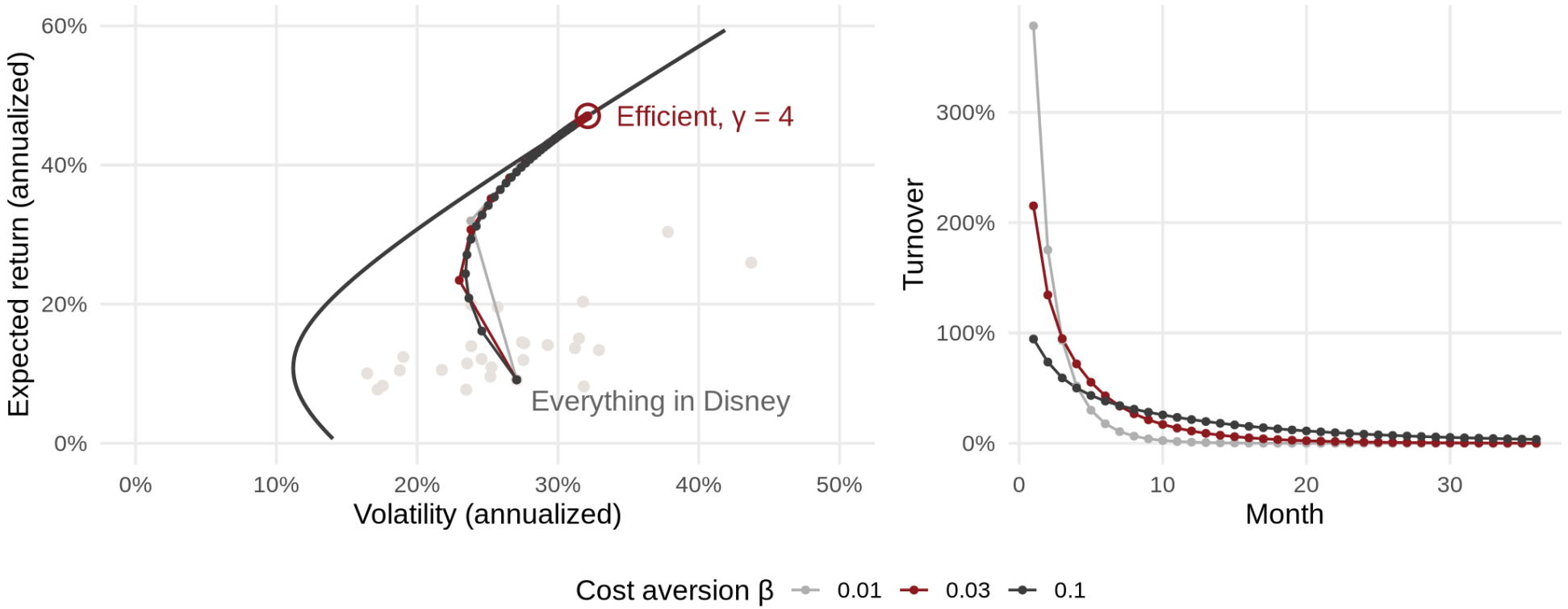
$$\omega_{t+1} = \arg\max_{\iota'\omega=1} \; \omega'\mu - \frac{\beta}{2}(\omega - \omega_{t^+})'(\omega - \omega_{t^+}) - \frac{\gamma}{2}\omega'\Sigma\omega$$

- The same problem as Markowitz, with

$$\Sigma^* = \Sigma + \frac{\beta}{\gamma} I \qquad \mu^* = \mu + \beta\, \omega_{t^+}$$

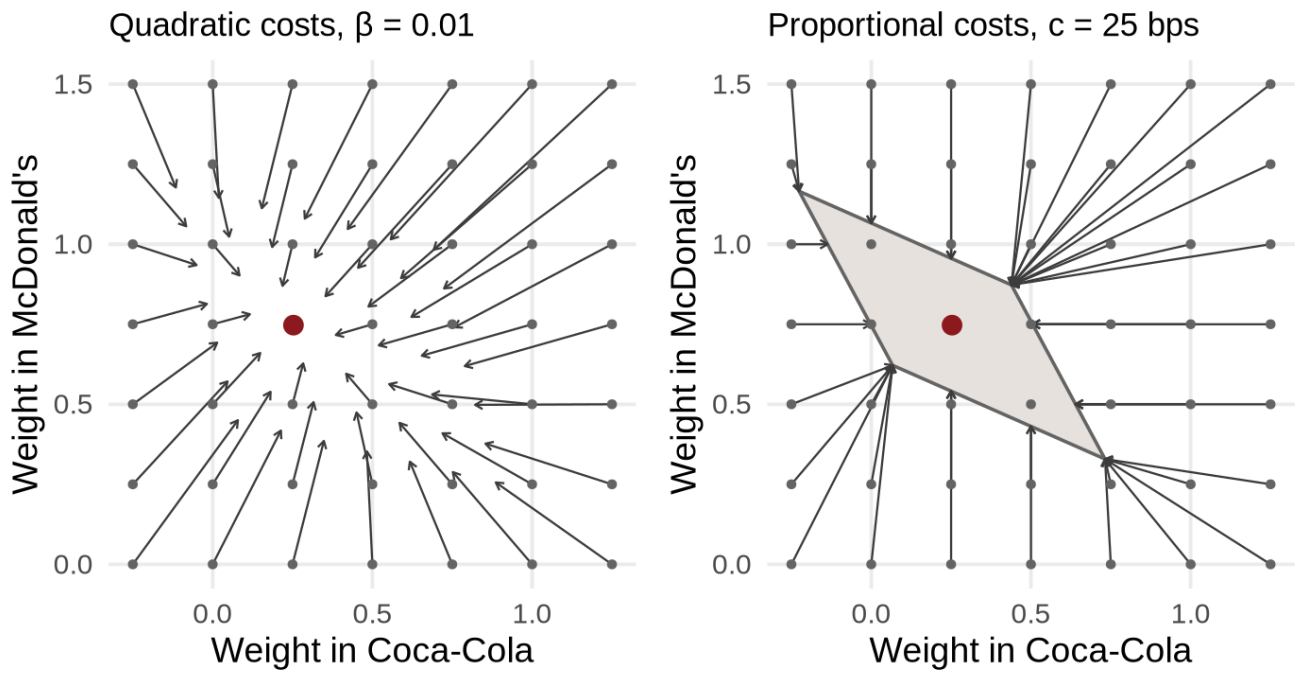
- **Costs act as shrinkage:** Σ^* shrinks the covariance matrix toward the identity, as in lecture 4, and μ^* pulls the weights toward the current portfolio
- $\beta = 0$: Markowitz. $\beta \to \infty$: no trading, $\omega_{t+1} = \omega_{t^+}$

Costs change the path, not the destination



- Known μ and Σ (lecture 2), $\gamma = 4$, no drift. Each month: the cost-aware weights given last month's portfolio
- Without costs, one trade of 748% of wealth (large long and short positions) gets there. With costs, the trade is spread over months: the larger β, the slower
- Every path ends at the efficient portfolio: first close to the frontier, then along it

Proportional costs: the no-trade zone



- Coca-Cola, McDonald's and cash for one month, $\gamma = 4$. Red: ω^*, the portfolio without costs; arrows: the trades
- **Quadratic costs:** every portfolio moves part of the way to ω^*
- **Proportional costs** $c\,\iota'|\omega - \omega_{t^+}|$, no closed form: inside the **no-trade zone** $|\gamma\Sigma(\omega - \omega^*)| \leq c$ no trade is worth its cost; outside, trade to its edge

Cost-aware weights in code

R

```r
# Hautsch & Voigt (2019): Markowitz with Sigma + beta / gamma * I and
# mu = beta * w_plus; mu = 0 otherwise, so no expected returns are estimated
cost_aware_weights <- function(sigma, w_plus, beta) {
  n <- ncol(sigma)
  sigma_star <- sigma + beta / gamma * diag(n)
  mu_star <- beta * w_plus
  sigma_inv <- solve(sigma_star)
  w_mvp <- rowSums(sigma_inv) / sum(sigma_inv)
  a <- sigma_inv - tcrossprod(rowSums(sigma_inv)) / sum(sigma_inv)
  as.vector(w_mvp + a %*% mu_star / gamma)
}
```

We choose the weights with the quadratic penalty and pay proportional costs on the trades

Cost-aware weights in code

Python

```python
# Hautsch & Voigt (2019): Markowitz with Sigma + beta / gamma * I and
# mu = beta * w_plus; mu = 0 otherwise, so no expected returns are estimated
def cost_aware_weights(sigma, w_plus, beta):
    n = len(w_plus)
    sigma_star = sigma + beta / gamma * np.eye(n)
    mu_star = beta * w_plus
    sigma_inv = np.linalg.inv(sigma_star)
    ones = np.ones(n)
    w_mvp = sigma_inv @ ones / (ones @ sigma_inv @ ones)
    a = sigma_inv - np.outer(sigma_inv @ ones, ones @ sigma_inv) / (ones @ sigma_inv @ ones)
    return w_mvp + a @ mu_star / gamma
```

We choose the weights with the quadratic penalty and pay proportional costs on the trades

Cost-aware weights without short sales

The same objective as a quadratic program: minimize $\frac{1}{2}\omega'(\gamma\Sigma + \beta I)\,\omega - \beta\,\omega_{t^+}'\omega$ subject to $\iota'\omega = 1$ and $\omega \geq 0$

R

```
cost_aware_weights_no_short <- function(sigma, w_plus, beta) {
  n <- ncol(sigma)
  solve.QP(
    Dmat = gamma * sigma + beta * diag(n), dvec = beta * w_plus,
    Amat = cbind(1, diag(n)), bvec = c(1, rep(0, n)), meq = 1
  )$solution
}
```

- $\beta = 0$: lecture 4's minimum variance with no short sales. $\beta \to \infty$: the weights stay at ω_{t^+}, buy and hold

Cost-aware weights without short sales

The same objective as a quadratic program: minimize $\frac{1}{2}\omega'(\gamma\Sigma + \beta I)\,\omega - \beta\,\omega'_{t+}\omega$ subject to $\iota'\omega = 1$ and $\omega \geq 0$

Python

```
def cost_aware_weights_no_short(sigma, w_plus, beta):
    n = len(w_plus)
    d = gamma * sigma + beta * np.eye(n)
    return minimize(
        lambda w: w @ d @ w / 2 - beta * w_plus @ w, x0=w_plus, jac=lambda w: d @ w - beta * w_plus,
        bounds=[(0, 1)] * n, method="SLSQP", options={"ftol": 1e-12},
        constraints={"type": "eq", "fun": lambda w: w.sum() - 1},
    ).x
```

- $\beta = 0$: lecture 4's minimum variance with no short sales. $\beta \to \infty$: the weights stay at ω_{t+}, buy and hold

The Dow Jones with costs

R

```
backtest_costs <- expand_grid(short_sales = c(TRUE, FALSE), beta = betas) |>
  pmap(\(short_sales, beta) {
    choose_weights <- if (short_sales) cost_aware_weights else cost_aware_weights_no_short
    w <- rep(1 / n_stocks, n_stocks)
    results <- vector("list", length(months))
    for (k in seq_along(months)) {
      t <- months[k]
      sigma <- cov(excess_matrix[(t - window_length):(t - 1), ])
      drift <- w * (1 + returns_matrix[t - 1, ])
      w_plus <- drift / sum(drift)
      w <- choose_weights(sigma, w_plus, beta)
      results[[k]] <- tibble(
        short_sales = short_sales, beta = beta, date = dates[t],
        ret_excess = sum(w * excess_matrix[t, ]), turnover = sum(abs(w - w_plus))
      )
    }
    bind_rows(results)
  }) |>
  bind_rows()
```

`returns_matrix` holds the returns, `excess_matrix` the excess returns, one column per stock; `betas` is a grid from 0 to 1000 (as floats)

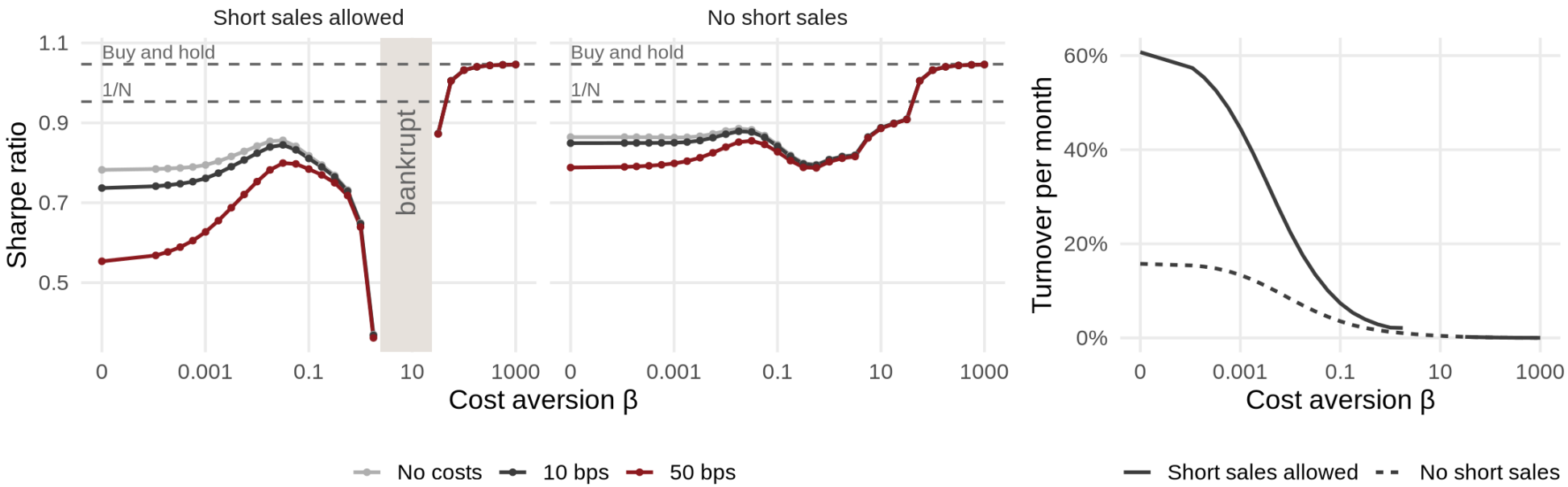
The Dow Jones with costs

Python

```
records = []
for short_sales in [True, False]:
    choose_weights = cost_aware_weights if short_sales else cost_aware_weights_no_short
    for beta in betas:
        w = np.full(n_stocks, 1 / n_stocks)
        for t in range(window_length, n_months):
            sigma = np.cov(excess_matrix[t - window_length:t], rowvar=False)
            drift = w * (1 + returns_matrix[t - 1])
            w_plus = drift / drift.sum()
            w = choose_weights(sigma, w_plus, beta)
            records.append({"short_sales": short_sales, "beta": beta, "date": dates[t],
                            "ret_excess": w @ excess_matrix[t], "turnover": np.abs(w - w_plus).sum()})
backtest_costs = pl.DataFrame(records)
```

`returns_matrix` holds the returns, `excess_matrix` the excess returns, one column per stock; `betas` is a grid from 0 to 1000 (as floats)

How much cost aversion?



- **Small β:** Sharpe ratio after 10 bps from 0.74 to 0.84 at $\beta \approx 0.03$, turnover from 61% to 13%
- **Large β:** no trading, buy and hold (1.05), which wins here only through the choice of stocks
- **Short sales allowed,** β from 3 to 18: too slow to close a short in NVIDIA while its price rises, bankrupt in 2024–2025

Parametric portfolio policies

Many stocks, few parameters

- The US stock market has thousands of stocks. Estimating μ and Σ for all of them is hopeless, even with shrinkage
- What we know about a stock is in its **characteristics**: size, value, momentum, profitability, …
- Brandt, Santa-Clara & Valkanov (2009): model the **weights** directly as a function of the characteristics, with one coefficient per characteristic for all stocks and all months

Which characteristics would you use?

In the lecture, you name the stock characteristics you would use to tilt a portfolio away from the market, and an AI groups them into a word cloud

Brandt, Santa-Clara & Valkanov (2009) use **size**, **book-to-market** and **momentum**. We use size and momentum from CRSP; book-to-market needs accounting data

Whiteboard: parametric portfolio policies

Tilt a benchmark $\bar{\omega}_{i,t}$ (e.g. value weights) with standardized characteristics $\hat{x}_{i,t}$:

$$\omega_{i,t} = \bar{\omega}_{i,t} + \frac{1}{N_t}\theta' \hat{x}_{i,t} \qquad \text{and choose} \qquad \max_{\theta} \frac{1}{T}\sum_{t} u\big(r_{p,t+1}(\theta)\big)$$

- θ is the same for every stock and month: **two parameters** for two characteristics, whether there are 30 or 5,000 stocks
- The characteristics have mean zero across stocks every month, so the tilts sum to zero and the weights to one
- No μ or Σ: the policy maximizes the average utility of its own returns, so skewness and fat tails count, too
- Costs and constraints (e.g. no short sales) can go into the objective

Characteristics from CRSP

R

```
crsp_monthly <- read_parquet(
  "data/crsp_monthly.parquet",
  col_select = c("permno", "date", "ret_excess", "mktcap", "mktcap_lag")
)
crsp_lags <- crsp_monthly |>
  transmute(permno, date = date %m+% months(13), mktcap_13 = mktcap)

policy_data <- crsp_monthly |>
  inner_join(crsp_lags, join_by(permno, date)) |>
  filter(date >= policy_start) |>
  transmute(
    permno, date, ret_excess, mktcap_lag,
    momentum = mktcap_lag / mktcap_13,
    size = log(mktcap_lag)
  ) |>
  drop_na()
```

Momentum: the change in market value from month $t-13$ to $t-1$, as in the Tidy Finance chapter **Parametric portfolio policies**; size: the log market value at $t-1$

Characteristics from CRSP

Python

```
crsp_monthly = pl.read_parquet(
    "data/crsp_monthly.parquet",
    columns=["permno", "date", "ret_excess", "mktcap", "mktcap_lag"],
)
crsp_lags = crsp_monthly.select(
    "permno", pl.col("date").dt.offset_by("13mo"), mktcap_13=pl.col("mktcap")
)

policy_data = (
    crsp_monthly.join(crsp_lags, on=["permno", "date"])
    .filter(pl.col("date") >= date(1980, 1, 1))
    .select(
        "permno", "date", "ret_excess", "mktcap_lag",
        momentum=pl.col("mktcap_lag") / pl.col("mktcap_13"),
        size=pl.col("mktcap_lag").log(),
    )
    .drop_nulls()
)
```

Momentum: the change in market value from month $t-13$ to $t-1$, as in the Tidy Finance chapter **Parametric portfolio policies**; size: the log market value at $t-1$

Standardized characteristics and benchmark weights

R

```
policy_data <- policy_data |>
  mutate(
    across(c(momentum, size), \(x) (x - mean(x)) / sd(x)),
    weight_benchmark = mktcap_lag / sum(mktcap_lag),
    n = n(),
    .by = date
  )
```

Every month, the characteristics have mean zero and standard deviation one across stocks; the benchmark is the value-weighted market

Standardized characteristics and benchmark weights

Python

```
policy_data = policy_data.with_columns(
    *[((pl.col(c) - pl.col(c).mean().over("date")) / pl.col(c).std().over("date")).alias(c)
      for c in ["momentum", "size"]],
    weight_benchmark=pl.col("mktcap_lag") / pl.col("mktcap_lag").sum().over("date"),
    n=pl.len().over("date"),
)
```

Every month, the characteristics have mean zero and standard deviation one across stocks; the benchmark is the value-weighted market

Estimating θ

R

```r
policy_returns <- function(theta, data) {
  data |>
    mutate(weight = weight_benchmark + (theta[1] * momentum + theta[2] * size) / n) |>
    summarize(ret = sum(weight * ret_excess), .by = date)
}

average_utility <- function(theta, data) {
  r <- policy_returns(theta, data)$ret
  mean((1 + r)^(1 - gamma_policy) / (1 - gamma_policy))
}

in_sample <- policy_data |> filter(date < policy_split)
fit <- optim(
  c(0, 0), \(theta) -average_utility(theta, in_sample),
  method = "Nelder-Mead", control = list(reltol = 1e-12, maxit = 1000)
)
theta <- fit$par
```

CRRA utility with γ = 5, estimated on 1980–2009: $\hat{\theta}_{\text{momentum}}$ = 0.48, $\hat{\theta}_{\text{size}}$ = -1.90. The policy tilts toward past winners and toward small stocks

Estimating θ

Python

```python
def policy_returns(theta, data):
    return (
        data.with_columns(
            weight=pl.col("weight_benchmark")
            + (theta[0] * pl.col("momentum") + theta[1] * pl.col("size")) / pl.col("n")
        )
        .group_by("date")
        .agg(ret=(pl.col("weight") * pl.col("ret_excess")).sum())
        .sort("date")
    )

def average_utility(theta, data):
    r = policy_returns(theta, data)["ret"].to_numpy()
    return np.mean((1 + r) ** (1 - gamma_policy) / (1 - gamma_policy))

in_sample = policy_data.filter(pl.col("date") < date(2010, 1, 1))
fit = minimize(lambda theta: -average_utility(theta, in_sample), x0=[0, 0],
               method="Nelder-Mead", options={"xatol": 1e-8, "fatol": 1e-14})
theta = fit.x
```

In sample and out of sample

Period	Portfolio	Excess return	Volatility	Sharpe ratio
In sample, 1980–2009	Value-weighted market	6.5%	15.8%	0.41
In sample, 1980–2009	Parametric policy	12.9%	19.6%	0.66
Out of sample, 2010–2025	Value-weighted market	13.1%	14.9%	0.88
Out of sample, 2010–2025	Parametric policy	9.0%	20.2%	0.45

- In sample, the policy beats the market (Sharpe ratio 0.66 against 0.41); out of sample it falls behind (0.45 against 0.88): after 2009, large stocks led
- Few parameters reduce estimation error, but they cannot protect against a world that changes. Out-of-sample evaluation again

Portfolio choice in five lectures

- **Mean-variance** gives the optimal portfolio, if we knew μ and Σ
- **Estimation error** makes plug-in portfolios fail out of sample; 1/N is hard to beat
- **Shrinkage, constraints and priors** trade a little bias for much less variance
- **Transaction costs** shrink, too, and make trading less often optimal
- **Parametric policies** replace N means and $N(N+1)/2$ covariances with a few coefficients

> Assignment 1 on portfolio choice starts today